7 CLOUD ARCHITECTURE PATTERNS EVERY ENGINEER SHOULD KNOW

These patterns power 90% of modern cloud applications and AI systems.

WHY SHOULD YOU CARE?

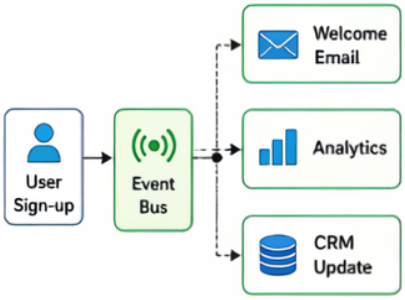
These are battle-tested strategies modern teams use to solve real-world challenges—handling millions of users, minimizing downtime, and evolving systems without a full rewrite.

More importantly, this is where most of your interview answers come from.

1 MICROSERVICES ARCHITECTURE

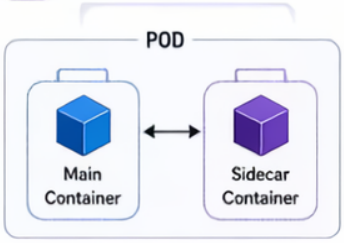
- Break large applications into smaller, independent services
- Each service handles a specific responsibility, communicates via APIs
- **Example:** Separate services for authentication, orders, and payments
- **When to use:** When you want independent deployments, scaling, and team ownership.

2 EVENT-DRIVEN ARCHITECTURE



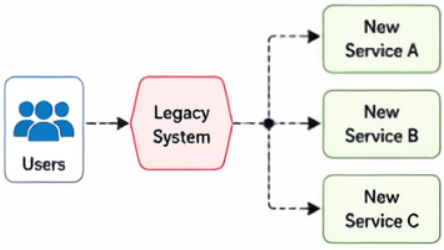
- Systems react to events asynchronosuly
- Events (like user sign-ups) trigger workflows automatically
- Reduces tight coupling between components
- **Example:** File upload → triggers processing + notifications
- **When to use:** For real-time systems or loosely coupled workflows

3 SIDECAR PATTERN



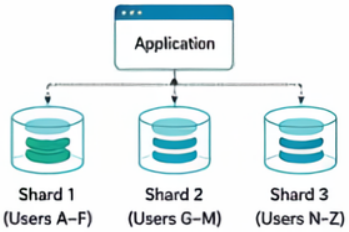
- Run a helper container alongside your main app
- Handles logging, service discovery, or networking
- Keeps your application focused on business logic
- **Example:** Service mesh sidecar managing traffic
- **When to use:** In Kubernetes or microservices environments

4 STRANGLER FIG PATTERN



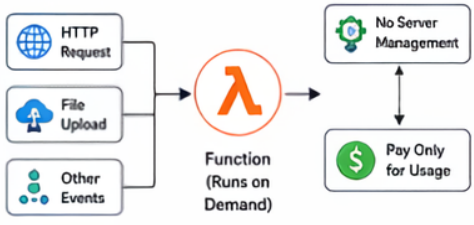
- Replace legacy systems gradually
- Route functionallity step by step from old system to new services
- **Example:** Migrating a monolith to microservices without downtime
- **When to use:** During legacy modernization with minimal risk

5 DATABASE SHARDING (HORIZONTAL SCALING)



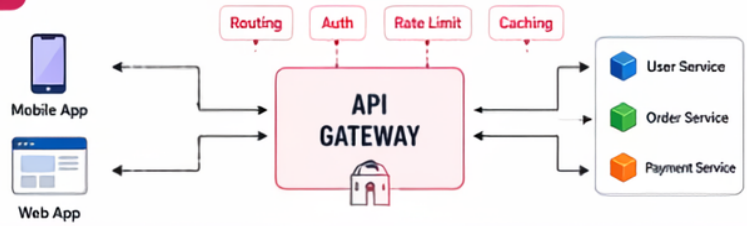
- Split data across multiple databases or nodes
- Each shard holds part of the dataset
- Improves performance and scalability
- **Example:** Users distributed by region or ID range
- **When to use:** For high-throughput or global-scale systems

6 SERVERLESS ARCHITECTURE



- Run code on demand without managing servers
- Functions are triggered by events (HTTP calls, file uploads, etc.)
- Pay only for what you use
- Example: Function triggered on API request or file upload
- When to use: For unpredictable workloads or rapid development cost-efficiency

7 API GATEWAY PATTERN



- Use a single entry point for all client requests
- Acts as a single entry point, handling routing, authentication, and throttling
- **Example:** Frontend calls one endpoint → gateway routes to multiple services
- **When to use:** To simplify client interaction and secure backend services

WHEN TO USE?

Choose the pattern based on your goals, system scale, team structure, and evolution roadmap.

PRO TIP

Most real-world systems use a combination of multiple patterns—not just one!